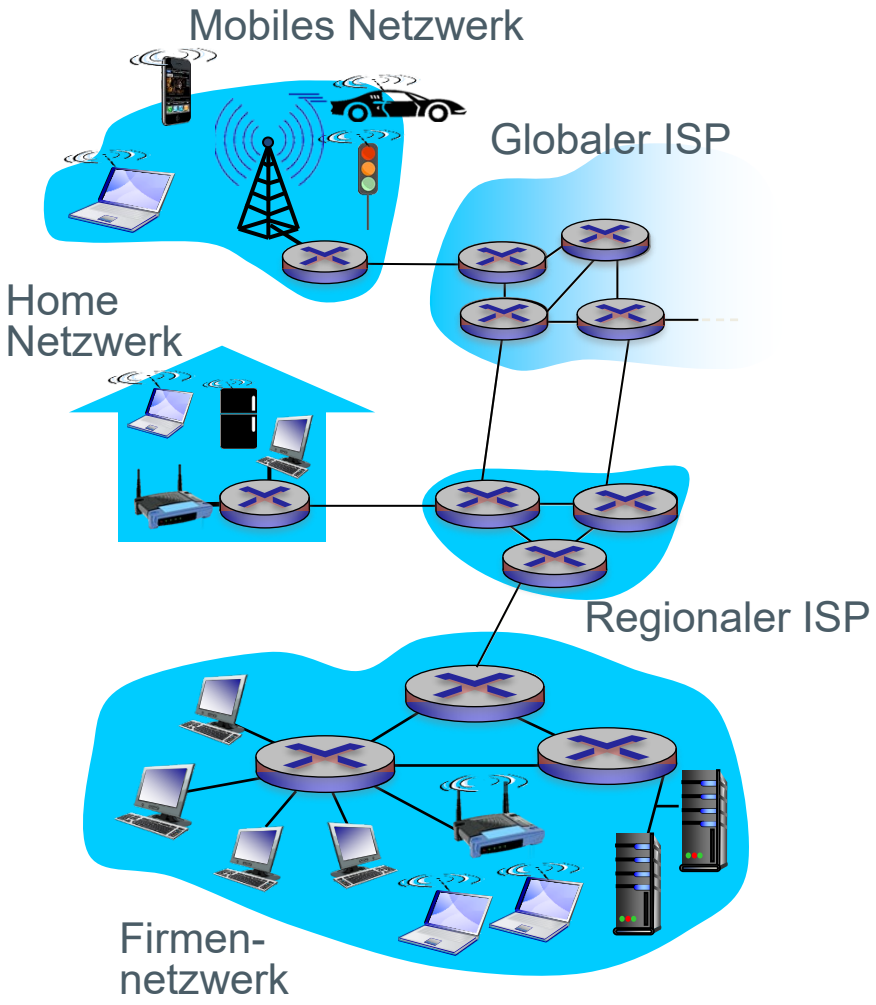
1.	Organisatorisches	4
2.	Einleitung	21
3.	HTML	46
4.	CSS	70
5.	Frameworks	89
6.	HTTP	106
7.	**Internet**	**126**
8.	TypeScript	178
9.	HTML+CSS+Typescript	224
10.	Web Services	235
11.	REST-Services	258

Woraus besteht das Internet?

- Internet = **Netzwerk** von Netzwerken
- **Protokolle**: kontrollieren das Senden und Empfangen von Nachrichten
- Internet **Standards**
  - RFC: Request for comments
  - IETF: Internet Engineering Task Force



ISP = Internet Service Provider

Komponenten von Netzwerken

Woraus bestehen Rechnernetze / „das Internet“ ?



Host

Dienen dem An-schluss des Hosts an ein Rechnernetz

Physikalische Verbindung

Daten werden in physikalische Signalfolgen übersetzt (z.B. Lichtsignale auf Lichtwellenleitern).
Achtung: Die eingesetzten Trägermedien sind fehleranfällig!

Infrastruktur-komponenten

Verbinden die Hosts untereinander, aber auch die Rechnernetze miteinander.
Im Internet geschieht dies durch die **Internet Service Provider** (ISPs).

Abstimmung

Heben Sie die offene Hand für „Ja“, die Faust für „Nein“.

1. Muss meine Anwendung (zB Browser) wissen, welche phys. Verbindungen(zum Beispiel Kabel oder WLAN) genutzt werden?

2. In Internet können Daten verlorengehen.

3. Dagegen kann man nichts tun.

Übertragungsrate

- Unter **Datenübertragungsrate** verstehen wir die maximale Menge von Daten, die in einer Zeiteinheit (z.B. eine Sekunde) übertragen werden können.
  Oft werden Datenübertragungsrate und Bandbreite synonym benutzt.
  Beispiel: DSL-Verbindung mit 250Mb/s.

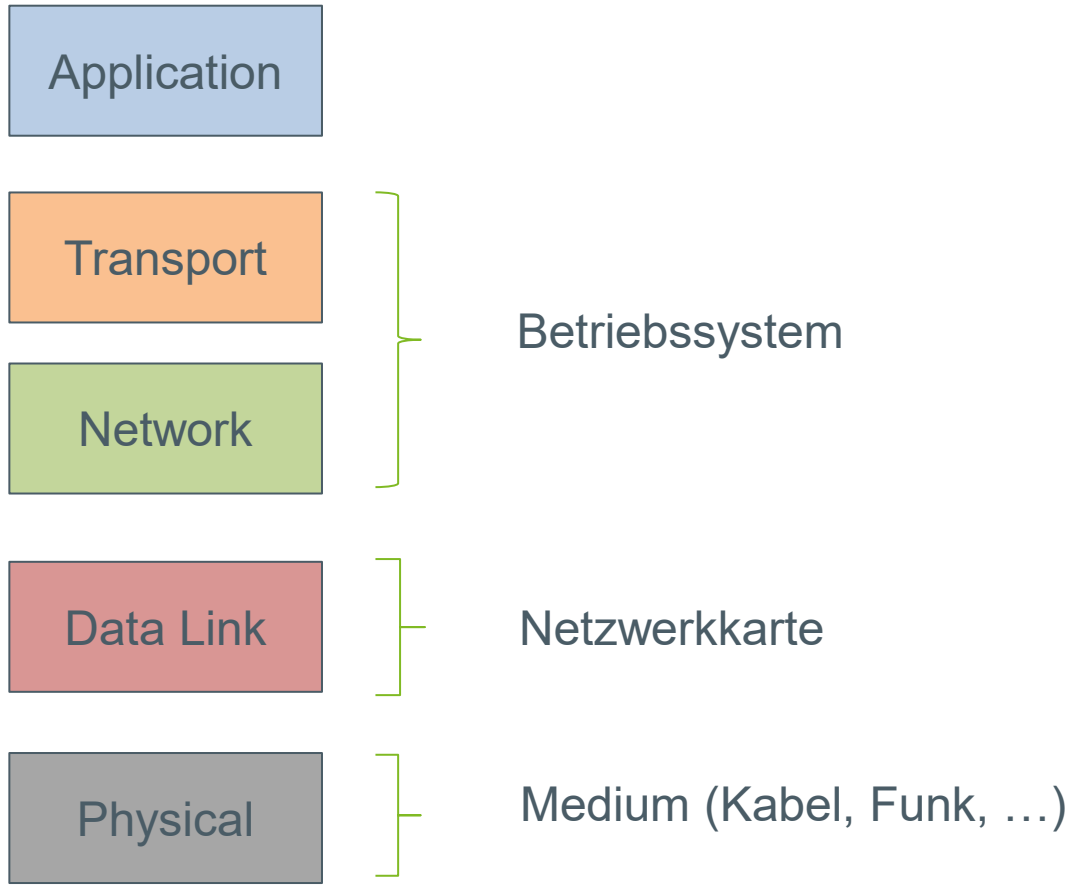
Übertragungsrate

- Nicht zu verwechseln mit **Latenz** (Verzögerung):
  Die Latenzzeit ist die Verzögerung, die während der Übertragung der Daten über das Netzwerk auftritt.
  Auf der gesamten Strecke vom Sender zum Empfänger kann es, wie wir noch sehen werden, zu Verzögerungen bei der Verarbeitung/Weiterleitung der Daten kommen.
  Zusätzlich wird Zeit für die Signalausbreitung benötigt. Diese hängt vom Trägermedium ab und liegt unterhalb der Lichtgeschwindigkeit.

Latenz

Das Internet ist in Schichten aufgebaut

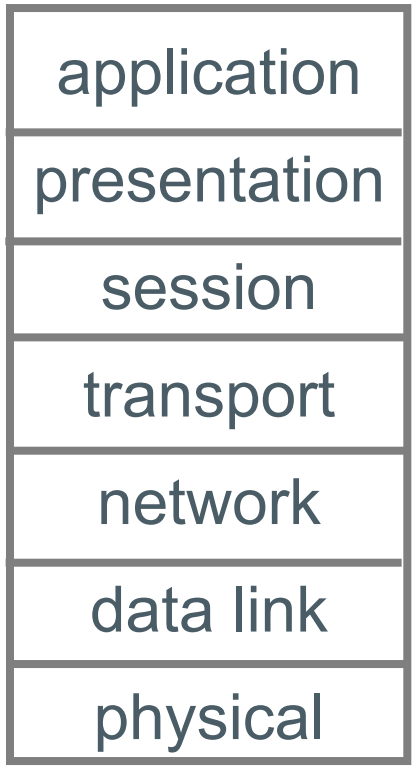
- **Anwendungsschicht** (*application layer*):
  Unterstützt die Anwendungen bei der Kommunikation
  - HTTP, SMTP, FTP, …
- **Transportschicht** (*transport layer*):
  Datentransfer zwischen “Prozessen”
  - TCP, UDP
- **Netzwerkschicht** (*network layer*):
  Routing der Pakete von der Quelle bis zum Ziel
  - IP, Routing Protokolle
- **Sicherungsschicht** (data *link layer*):
  Datenübertragung zwischen benachbarten Netzwerkkomponenten
  - Ethernet, 802.11 (WiFi), PPP
- **Physische Schicht** *(physical layer)*:
  Elektrische Signale auf der Verbindung



ISO-OSI Referenzmodell ist die Grundlage

- **Präsentationsschicht** *(presentation layer):* Erlaubt es Anwendungen, die Daten zu interpretieren, z.B. durch Komprimierung, Verschlüsselung, …
- **Sitzungsschicht** *(session layer):* Synchronisation und Wiederherstellung von Daten.

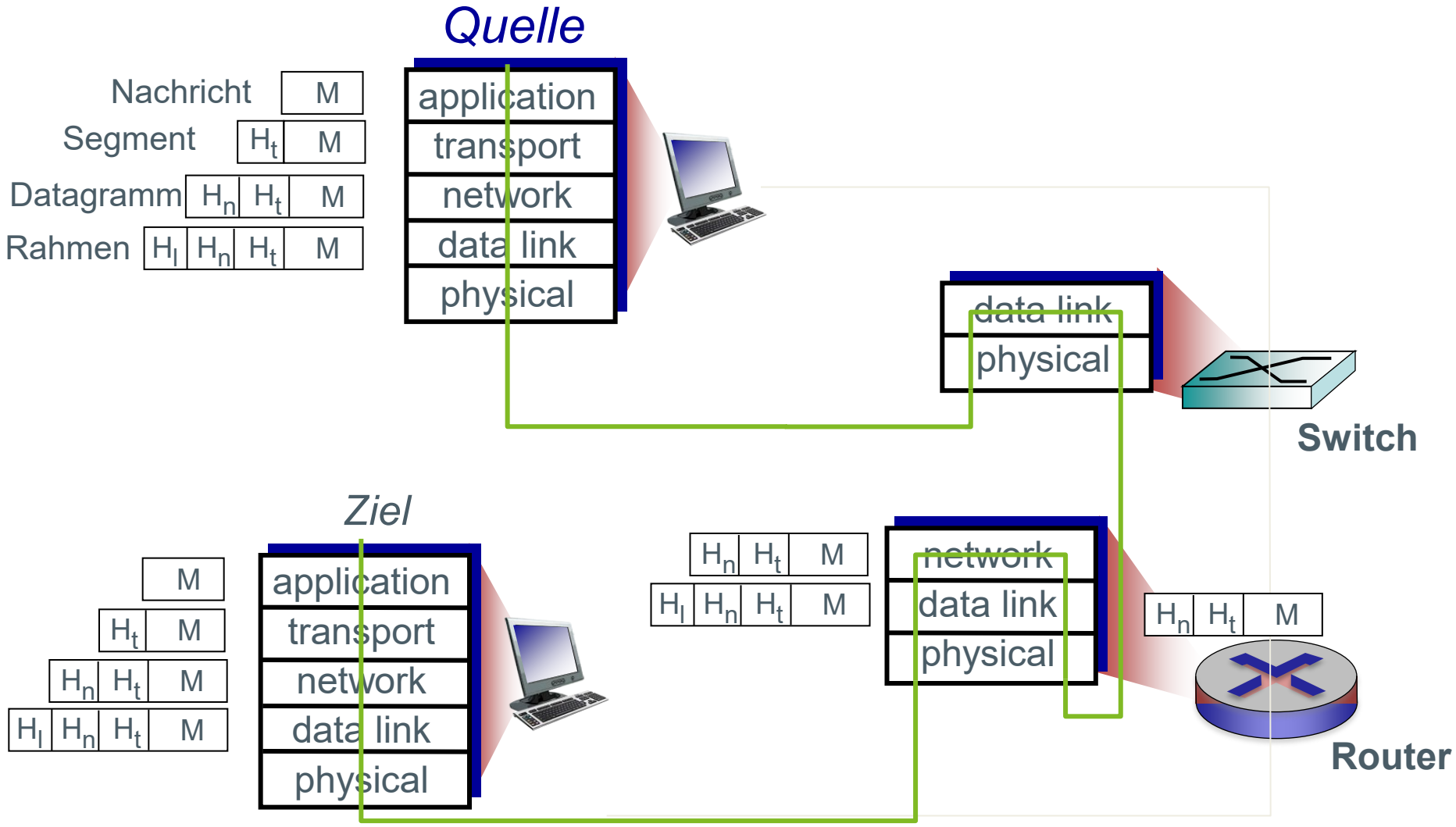
- Im Internet-Stack gibt es diese beiden Schichten nicht! Sie müssen daher, **wenn benötigt**, in der Applikation implementiert werden.



ISO/OSI-Modell:
7 Schichten

Merkspruch:
***P**lease*
***d**o*
***n**ot*
***t**hrow*
***s**alami*
***p**izza*
***a**way*

Beispiel


Quelle
Nachricht M
Segment Ht M
Datagramm Hn Ht M
Rahmen Hl Hn Ht M
application
transport
network
data link
physical
data link
physical
Switch
Ziel
M
Ht M
Hn Ht M
Hl Hn Ht M
application
transport
network
data link
physical
Hn Ht M
Hl Hn Ht M
network
data link
physical
Hn Ht M
Router

Internet-Stack bietet zwei Möglichkeiten für den Datentransport

TCP

- Verlässlicher Transport zwischen sendendem und empfangendem Prozess
- Verbindungsorientiert: Client und Server müssen zu Beginn Informationen austauschen
- Nicht: Garantierte Übertragungszeiten, Durchsatz, Sicherheit

UDP

- Unzuverlässiger Transport zwischen sendendem und empfangendem Prozess
- Verbindungslos
- Nicht: Garantierte Übertragungszeiten, Durchsatz, Sicherheit

Frage: Wozu UDP?

TCP: Transmission Control Protocol

UDP: User Datagram Protocol

UDP – Verbindungslose Kommunikation

- UDP ist die Low-Cost Airline im Internet!
  - „Best Effort“
    Keine Garantie, dass Segmente ankommen oder in der richtigen Reihenfolge ankommen
  - Verbindungslos
    - Keine initiale Abstimmung zwischen Client und Server
    - Jedes Segment ist unabhängig von den anderen

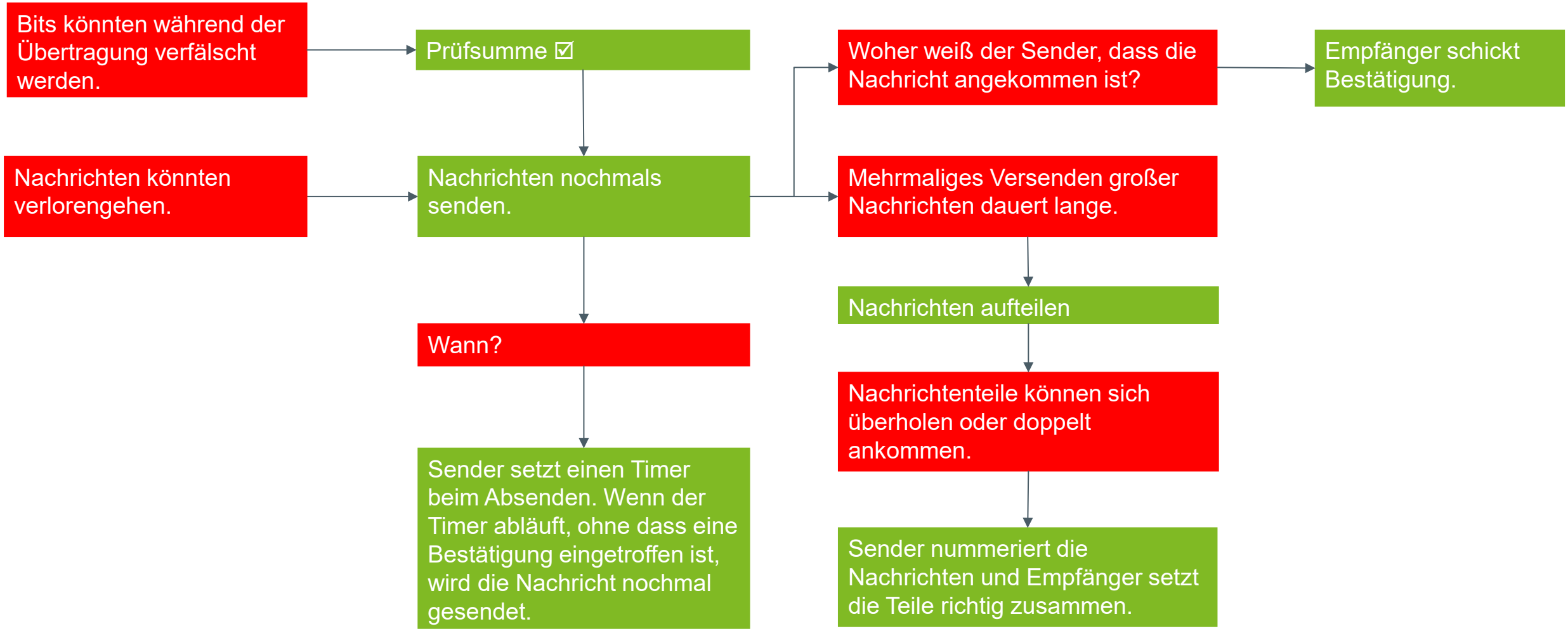
- Vorteile von UDP
  - Schneller, da initiale Abstimmung entfällt
  - Einfacher, da kein Zustand gespeichert wird
  - Geringe Header-Größe
  - Keine Überlastkontrolle wie bei TCP

TCP

- TCP ist „verlässlich“
  - TCP erstellt eine Verbindung zwischen einem Client und einem Server (**Punkt-zu-Punkt**)
  - Client und Server führen zu Beginn einen Handshake durch. TCP ist daher **verbindungsorientiert**
  - Kommunikation ist in beide Richtungen möglich
  - Aus Sicht der Anwendungen werden Byteströme übertragen, die auf der anderen Seite in der richtigen Reihenfolge ankommen.

- Wenn möglich, wollen wir
  - Verlässlichkeit: Alle Bytes kommen an – Achtung: Keine Garantie!
  - Reihenfolge: Alle Bytes kommen in der richtigen Reihenfolge an

Wie funktioniert TCP?



TCP Details

Bits könnten während der Übertragung verfälscht werden.

Jedes Segment enthält eine **Prüfsumme**

Woher weiß der Sender, dass die Nachricht angekommen ist?

Der Empfänger speichert die Nachrichten und sendet eine Bestätigung (ACK, „acknowledge“), welches eine **ACK-Nummer** enthält. Diese ist die Nummer des als nächsten erwarteten Bytes (!).

Nachrichten könnten verlorengehen.

Nachrichten nochmals senden.

Mehrmaliges Versenden großer Nachrichten dauert lange.

Nachrichten werden in **Segmente** aufgeteilt, die max. 1500 Bytes groß sind. Typischerweise werden mehrere Segmente gleichzeitig geschickt.

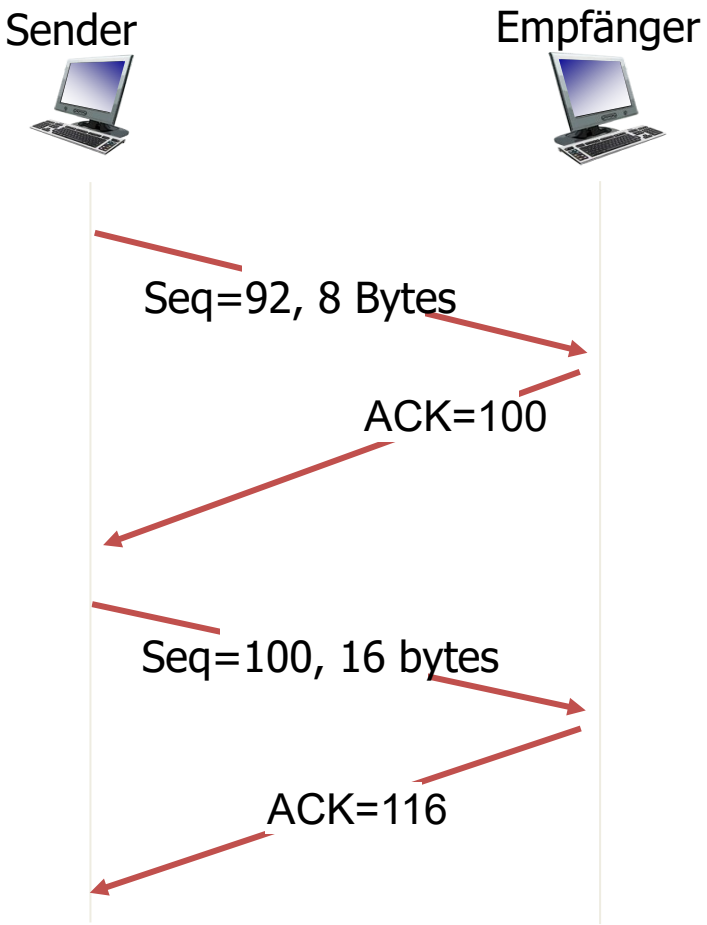
Wann?

Der Sender startet einen **Timer** für das älteste unbestätigte Segment . Bei Ablauf wird das Segment nochmal gesendet.

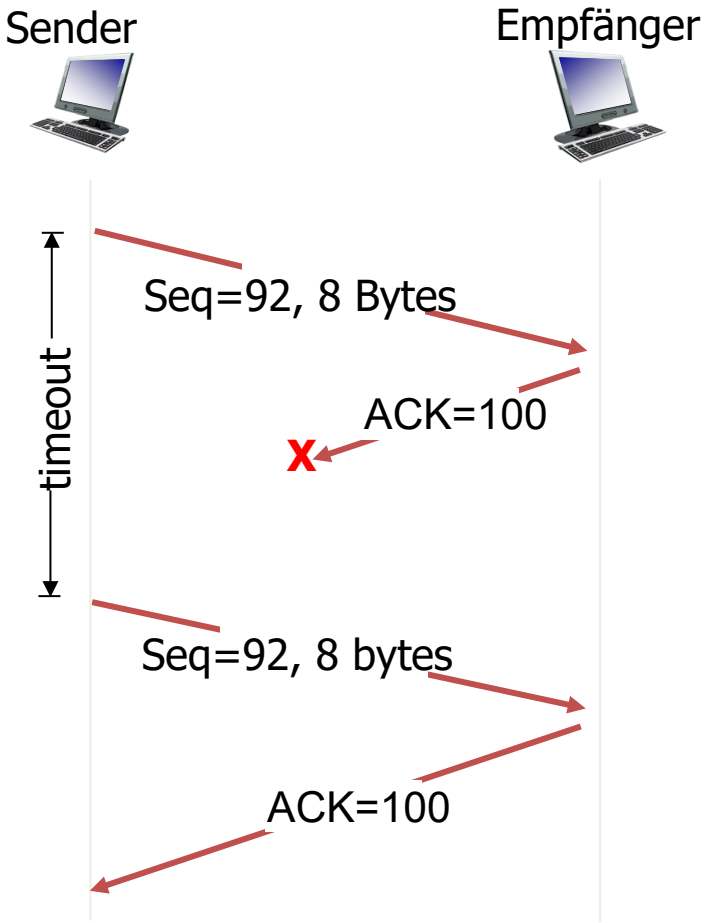
Bei Empfang eines ACKs, werden Segmente mit einer Sequenznummer kleiner als die ACK-Nummer als bestätigt vermerkt. Für das dann älteste unbestätigte Segment wird ein Timer gestartet.

Jedes Segment enthält **Sequenznummer**, die die Nummer des ersten Bytes in der Gesamtnachricht ist.

Beispiele: Ablauf ohne Störungen

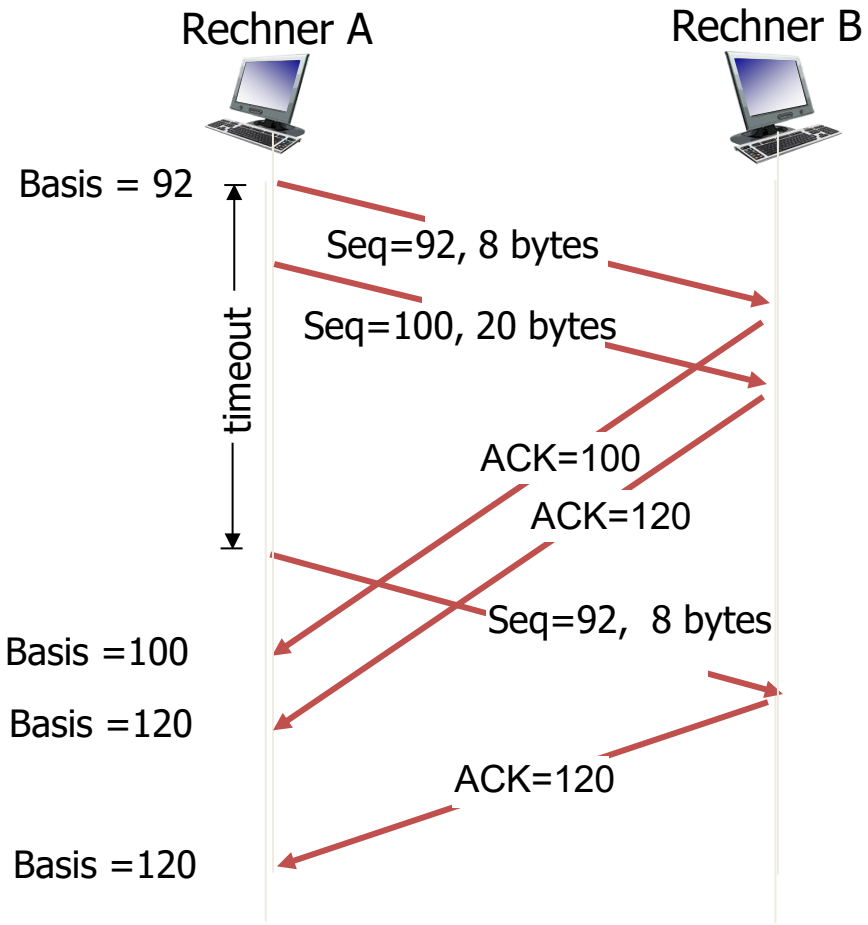


Beispiele: Verlorengegangenes ACK



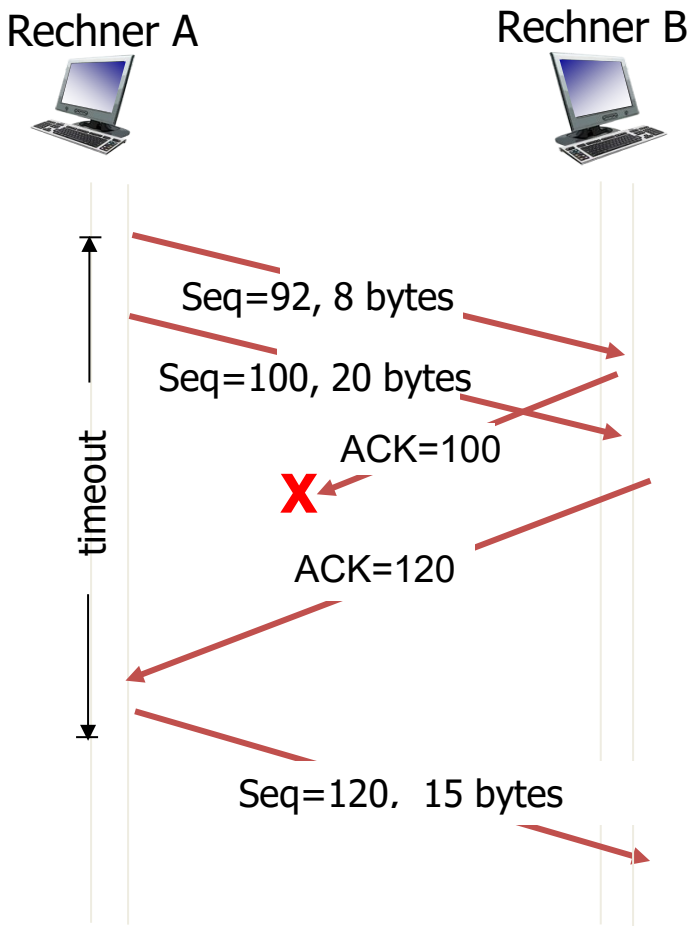
Nach dem timeout wird das Paket nochmals gesendet.

Beispiele: Vorzeitiges Timeout



Der Empfänger ignoriert das bereits erhaltene Segment mit der Nummer 92 und schickt erneut ACK 120

Beispiele: Kumulatives ACK



ACK 120 trifft ein, bevor der timer abläuft. Dadurch weiß der Sender, dass der Empfänger die ersten beiden Pakete korrekt empfangen hat. Kein erneutes Senden!

TCP-Verbindungen

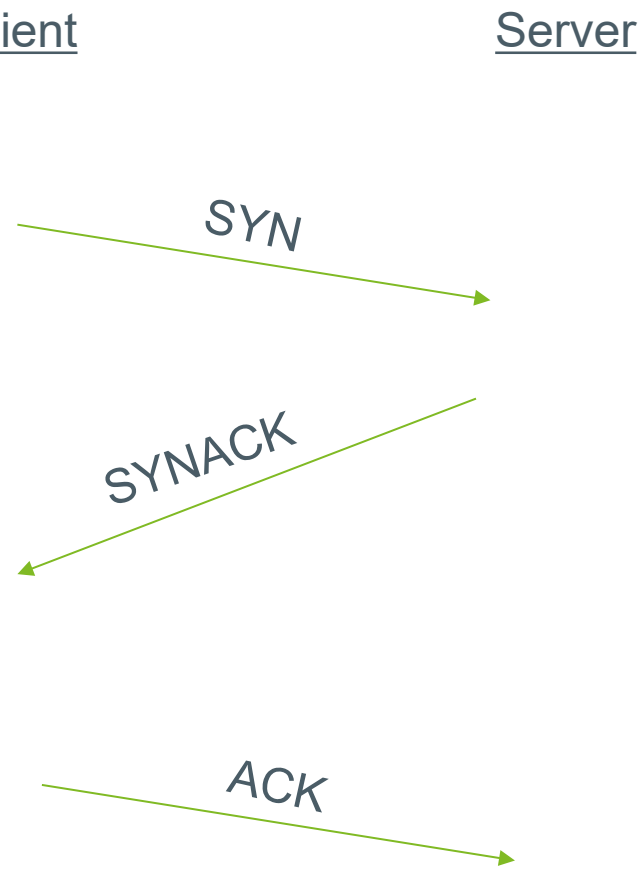
- „TCP ist verbindungsorientiert“ – was heißt das?

3-Wege Handshake:

1. Sender schickt zuerst ein kleines TCP-Segment, in dem keine Nutzdaten sind, sondern das SYN-Bit – siehe vorige Folie gesetzt ist. Außerdem fügt er eine zufällige initiale Sequenznummer hinzu (client_isn).

2. Nach Erhalt dieses Segments baut der Empfänger den Puffer auf und sendet ein Segment (SYNACK) zurück, in dem SYN gesetzt ist, die Acknowledge-Nummer wird auf client_isn+1 gesetzt und eine server_isn einfügt.

3. Sender allokiert beim Erhalt ebenfalls die Puffer und sendet ein Segment, das das SYNACK bestätigt, SYN=0. Dieses Segment kann bereits Daten enthalten.



QUIC

QUIC ist ein **verbindungsorientiertes** Protokoll auf Basis von UDP

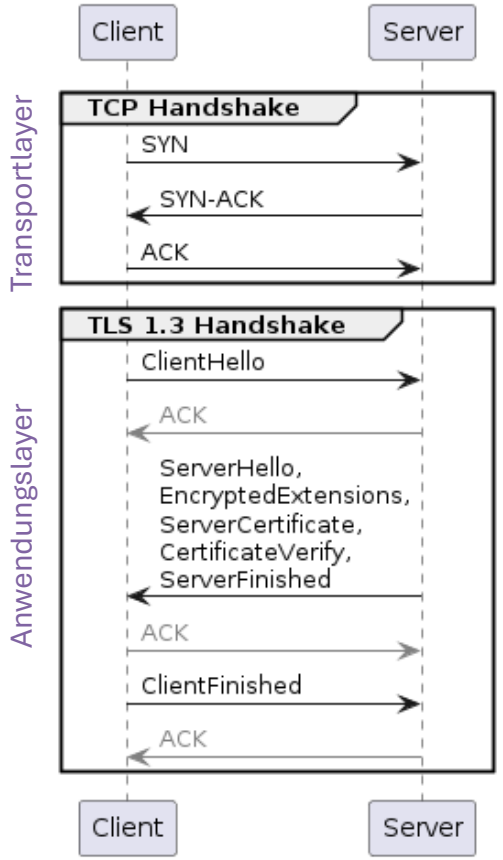
- Ermöglicht zuverlässige, schnelle und flexible Datenübertragung
- Kein klassischer „Handshake“ wie bei TCP erforderlich
- Integrierte Sicherheits- und Optimierungsfeatures
  - TLS 1.3 direkt eingebaut, keine zusätzliche Verschlüsselungsschicht notwendig
  - Multiplexing: Mehrere Datenströme können parallel übertragen werden
  - Basierend auf UDP für mehr Geschwindigkeit
- Schneller, sicherer und effizienter als TCP

- Benutzt von: HTTP(S)/3

ⓘ Ursprünglich von Google eingeführt, seit 2021 als RFC 9000 standardisiert.

Handshakes mit Verschlüsselung

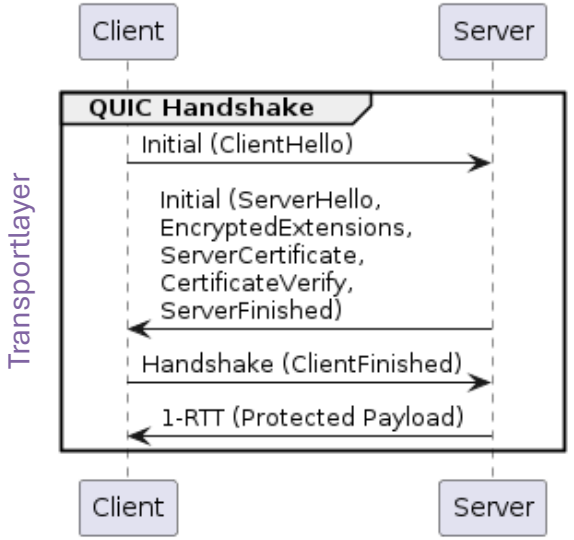
TCP und TLS 1.3 (bis HTTP/2)



- ☺ Reduzierter Overhead im Vergleich zu alten SSL/TLS Protokollen
- ☹ Separate TCP + TLS Handshakes
- ☹ ACKs können nicht vermieden werden, da TCP nicht weiß, dass es der TLS Handshake ist

➔ 9 TCP-Pakete

QUIC mit TLS 1.3 (ab HTTP/3)

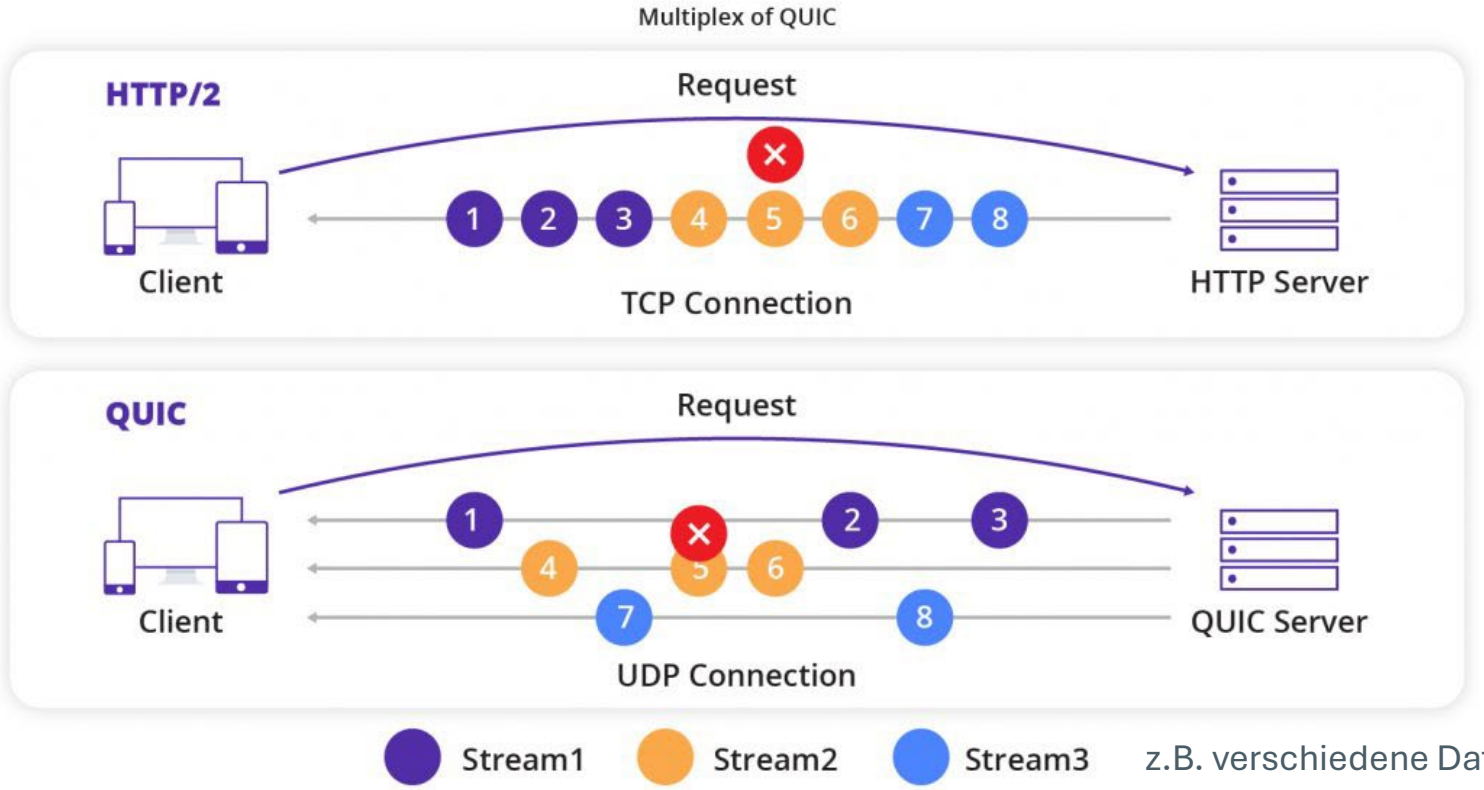


- ☺ Kombinierter Handshake vermeidet unnötige ACKs = Geringere Datenmenge
- ☺ Schnellerer Verbindungsaufbau = Geringere Latenz

➔ 4 UDP/QUIC-Pakete
➔ 0-RTT möglich

Multiplexing

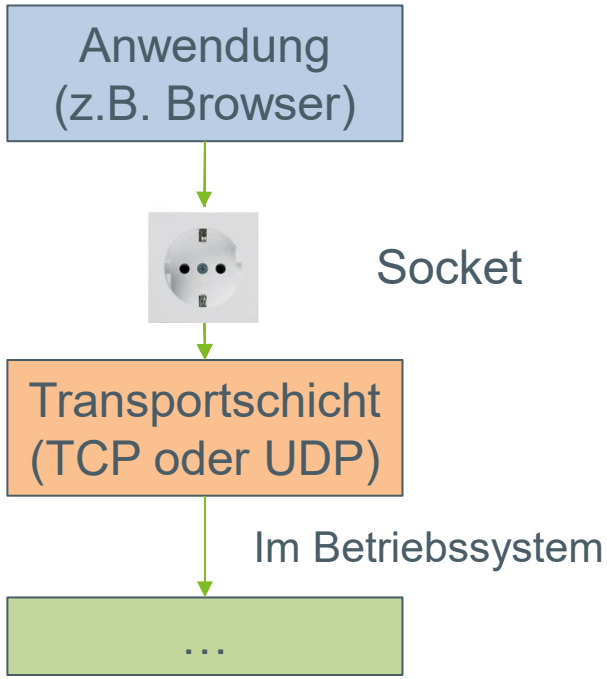
QUIC kann mehrere **Streams** (für Eingaben und Ausgaben) über die gleiche Verbindung erstellen
→ Weniger Head-of-Line Blocking (ein Paketverlust ✕ blockiert den kompletten Datenaustausch)



z.B. verschiedene Dateien (HTML, CSS, ...)

[Bildquelle: https://www.cdnetworks.com/wp-content/uploads/2023/03/QUIC-PICTURE-05-1024x560.jpg]

Verbindung Anwendungsschicht - Transportschicht



Ein **Socket** ist die Verbindung zwischen Anwendung und Transportschicht und hat eine eindeutige Nummer, die auch als Port-Nummer bezeichnet wird.

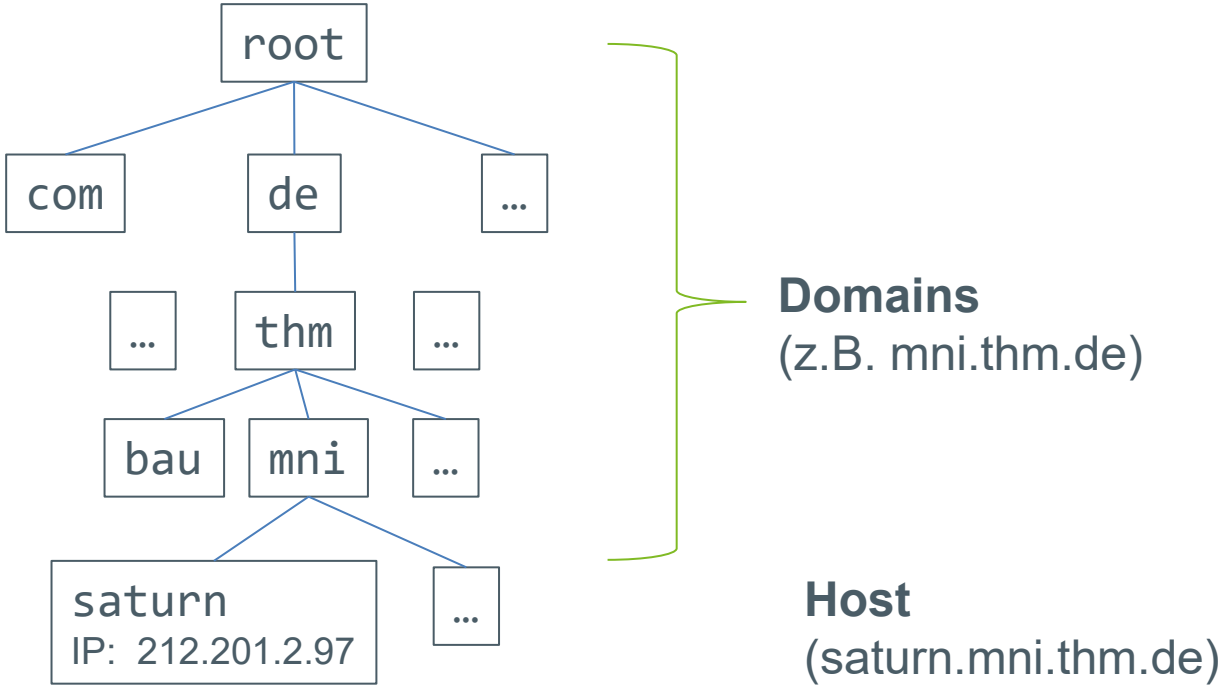
Die Port-Nummer kann der Programmierer der Anwendung vergeben.

Server-Anwendungen nutzen häufig Standard-Ports:
http: 80, email: 25, ftp: 20, …
https://de.wikipedia.org/wiki/Liste_der_standardisierten_Ports .

Beim „Öffnen“ eines Sockets gibt man den Hostnamen (oder die IP-Adresse) und die Port-Nummer mit.

Domain Naming System (DNS)

- 1. Problem: Woher weiß „das Internet“, in welchem Netzwerk sich der angesprochene Host befindet?
- 2. Problem: Wie findet man die Adresse bei Milliarden von Hosts schnell?



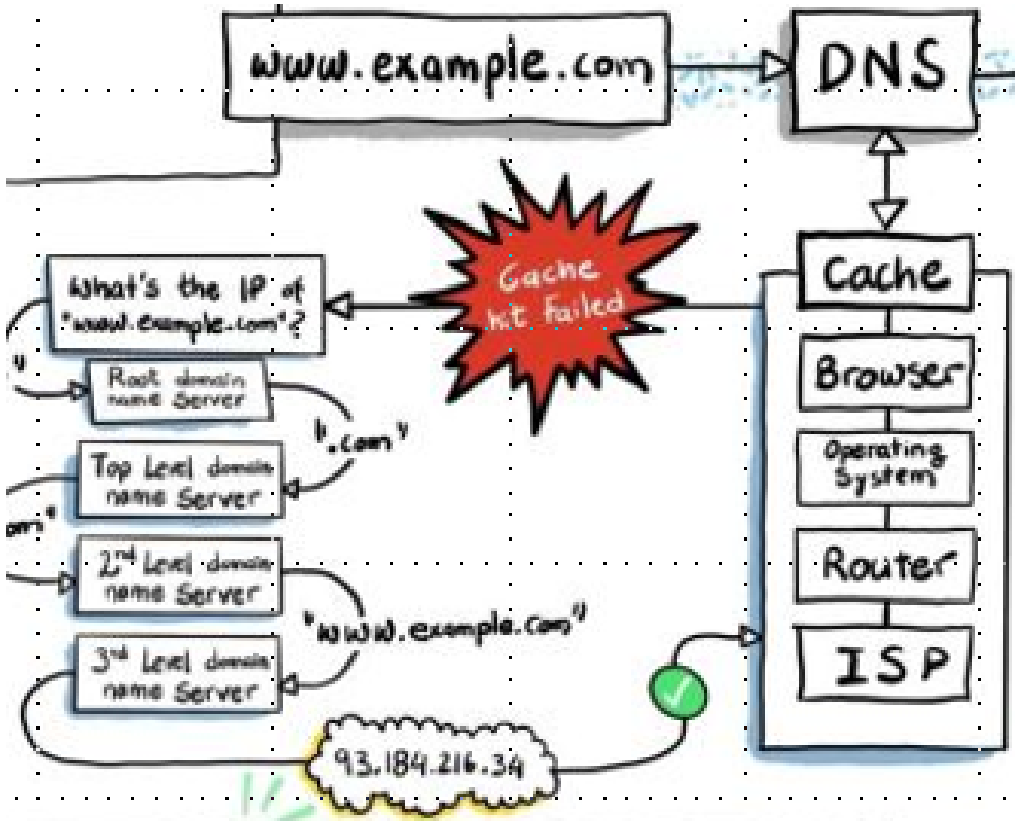
Domains
(z.B. mni.thm.de)

Host
(saturn.mni.thm.de)

Jede Domain hat einen oder mehrere **Name Server**.

Nutzung DNS

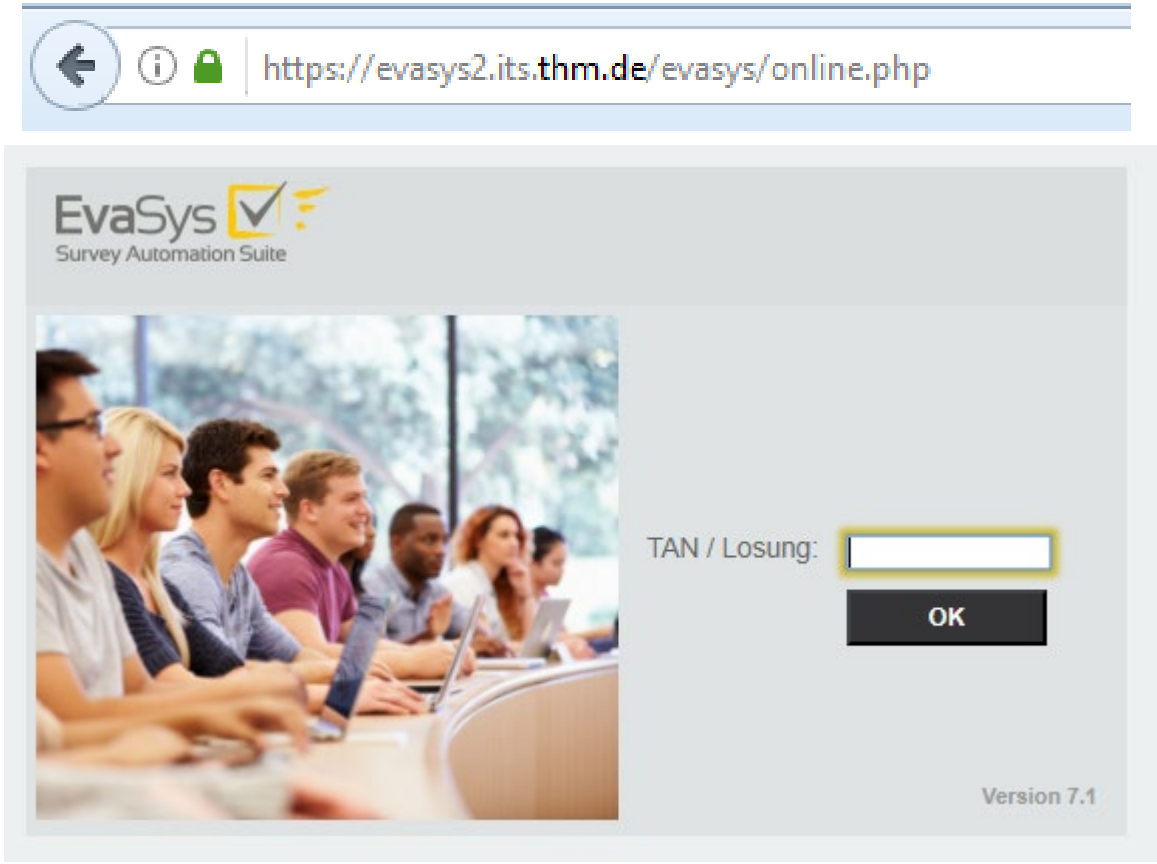
- Jedes Betriebssystem stellt Funktionen bereit, mit denen man das DNS anfragen kann.
- Sobald Sie Ihren Rechner/Handy/… mit einem Netzwerk verbinden, wird ihm ein Name Server zugewiesen.
- Probieren Sie bitte jetzt das Kommando `nslookup aus.`
  Wie heißt Ihr Name Server?
- Geben Sie irgendeinen Namen eines Hosts ein (z.B. `www.thm.de`).
  Wie lautet die Adresse?
- Jedes Mal, wenn Sie einen Hostname eingeben, wird das DNS gefragt. Die Ergebnisse werden eine Zeitlang zwischengespeichert.



Evaluationsverfahren „Online in Präsenz“ (OIP)

Link:
http://go.thm.de/eval

Losung:___74Q26_____



https://evasys2.its.thm.de/evasys/online.php?p=74Q26